X C P C T e m

BHUACM

2 0 2年 1 0月 1 8日

目 ⋮

1 动态规划

1.1 01 背包

```
for(int i = 1;i <= n;i++)
{
    for (int j = m;j >= w[i];j--)
        dp[j] = max(dp[j],dp[j - w[i]] + v[i]);
}
```

1.2 SOSDP

```
//SOSDP(子集和DP) x二进制长度为t dp[x] 从 x 的所有二进制子集转移过来

for(int i = 0;i < t;i++)
{
    for(int j = 0;j < (1 << t);j++)
    {
        if(j >> i & 1)
        {
            dp[j] += dp[j ^ (1 << i)];
        }
    }
}
```

1.3 m 元单调上升子序列

```
// m元单调上升子序列
cin >> n >> m;
for (int i = 1;i <= n;i++)
{
    cin >> arr[i];
    brr[i] = arr[i];
}
sort(brr+1,brr+1+n);
n1 = unique(brr+1,brr+1+n) - (brr+1);
for (int i = 1;i <= n;i++)
{
    arr[i] = lower_bound(brr+1,brr+1+n1,arr[i]) - brr;
    dp[1][i] = 1;
}
for (int i = 2;i <= m;i++)
{
    memset(t,0,sizeof(t));
    for (int j = 1;j <= n;j++)
    {
        dp[i][j] = ask(arr[j]-1);
        add(arr[j],dp[i-1][j]);
    }
}
ll ans = 0;
```

```
for (int i = 1;i <= n;i++)
    ans = (ans + dp[m][i]) % mod;
```

1.4 二进制优化多重背包

```
for (int i = 1;i <= n;i++)
{
    int w,v,p;
    cin >> w >> v >> p;
    for (int j = 1;j <= p;j = j << 1)
    {
        arr.push_back({j*w,j*v});
        p -= j;
    }
    if(p)
        arr.push_back({p*w,p*v});
}
for (int i = 0;i < arr.size();i++)
{
    for (int j = m;j >= arr[i].first;j--)
        dp[j] = max(dp[j],dp[j - arr[i].first] + arr[i].second);
}
```

1.5 全错排

```
dp[1] = 0;
dp[2] = 1;
for (int i = 3;i <= 20;i++)
{
    dp[i] = (i-1) * (dp[i-1] + dp[i-2]);
}
```

1.6 区间 dp

```
for (int len = 2;len <= n;len++)
{
    for (int l = 1;l + len - 1 <= n;l++)
    {
        int r = l + len - 1;
        for(int k = l;k < r;k++)
    }
}
```

1.7 四边形不等式优化 dp

```
for (int k = l;k < r;k++)
    dp[l][r] = min(dp[l][k] + dp[k+1][r]) + w(l,r);
   // 优化为:
```

```
for (int k = m[l][r-1];k <= m[l+1][r];k++)
    dp[l][r] = min(dp[l][k] + dp[k+1][r]) + w(l,r);
```

1.8 多重背包

```
for (int i = 1;i <= n;i++)
{
    for (int j = 1;j <= m;j++)
    {
        for (int k = 0;k < s[i];k++)
        {
            if(j < k*w[i])
                dp[i][j] = dp[i-1][j];
            else
                dp[i][j] = max(dp[i-1][j],dp[i-1][j - k*w[i]] + k*v[i]);
        }
    }
}
```

1.9 完全背包

```
for (int i = 1;i <= n;i++)
{
    for (int j = w[i];j <= m;j++)
        dp[j] = max(dp[j],dp[j - w[i]] + v[i]);
}
```

1.10 数位 dp

```
//limit 当前位限制 lead 前导0
ll ans[20];
ll arr[50];
ll dp[50][2][2][50];
int digit;
ll dfs(int pos,int limit,int lead,int sum)
{
    ll ans = 0;
    if(!pos)
        return sum;
    if(dp[pos][limit][lead][sum] != -1 && !limit && !lead)
        return dp[pos][limit][lead][sum];
    for (int i = 0;i <= (limit ? arr[pos] : 9);i++)
    {
        if(lead && i == 0)
            ans = ans + dfs(pos-1,limit && i == arr[pos],1,sum);
        else
            ans = ans + dfs(pos-1,limit && i == arr[pos],0,sum + (i == digit));
    }
    if(!lead && !limit)
        dp[pos][limit][lead][sum] = ans;
```

```
    return ans;
}
ll f(ll x)
{
    memset(dp,-1,sizeof(dp));
    int cnt = 0;
    while(x)
    {
        arr[++cnt] = x % 10;
        x /= 10;
    }
    return dfs(cnt,1,1,0);
}
void solve()
{
    ll a,b;
    cin >> a >> b;
    cout << f(b) - f(a-1) << ' ';
}
```

1.11 最大子段和

```
//mx[i] 代表考虑前i个数的最大子段和
for (int i = 1;i <= n;i++)
{
    dp[i] = max(dp[i-1] + arr[i],arr[i]);
    mx[i] = max(mx[i-1],dp[i]);
}
```

1.12 最长上升子序列

```
int len = 0; //len是答案
for (int i = 1;i <= n;i++)
{
    if(dp[len] < arr[i])
        dp[++len] = arr[i];
    else
        *lower_bound(dp+1,dp+len+1,arr[i]) = arr[i];
}
//要求完整的子序列可以权值线段树维护dp
```

1.13 最长公共子串

```
for (int i = 1; i <= n1; ++i)
    for (int j = 1; j <= n2; ++j)
        if (s1[i - 1] == s2[j - 1])
            dp[i][j] = dp[i - 1][j - 1] + 1;
        else
            dp[i][j] = 0;

```

```
for (int i = 1; i <= n1; ++i)
    for (int j = 1; j <= n2; ++j)
        ma = max(ma, dp[i][j]);
```

1.14 最长公共子序列

```
for(int i = 1;i <= n;i++)
{
    for(int j = 1;j <= m;j++)
    {
        if(s1[i] == s2[j])
            dp[i][j] = max(dp[i][j],dp[i-1][j-1] + 1);
        else
            dp[i][j] = max(dp[i-1][j],dp[i][j-1]);
    }
}
```

2 图论

2.1 01BFS

```
deque<int> q;
q.push_front(s);
dis[s] = 0;
while(!q.empty())
{
    int x = q.front();
    q.pop_front();
    if(vis[x]) continue;
    vis[x] = 1;
    for(auto it : g[x])
    {
        int to = it.to;
        int w = it.w;
        if(dis[to] > dis[x] + w)
        {
            dis[to] = dis[x] + w;
            if(w)
                q.push_back(to);
            else
                q.push_front(to);
        }
    }
}
```

2.2 Bellman-ford

```
ll dis[MAXN],last[MAXN],re[MAXN] //dis距离，last用来存上一次的结果。
for (int i = 1;i <= n;i++)
{
     for (int j = 1;j <= n;j++)
        re[i][j] = INF;
}
void Bellman_ford()
{
    for (int i = 1;i <= n;i++)
        dis[i] = INF;
    dis[1] = 0;
    for (int i = 1;i < n;i++)
    {
        memcpy(last,dis,sizeof dis);
        for (int j = 1;j <= m;j++)
        {
            ll fr = edges[j].fr,to = edges[j].to,w = edges[j].w;
            dis[to] = min(dis[to],last[fr] + w);
        }

        // for (int j = 1;j <= n;j++)
        //     re[i][j] = dis[j];
        // i 是边数
        // re数组维护了从 1 到 j 使用了 i 条边的最短路
    }
        // for (int j = 1;j <= cnt;j++)
        // {
        //     ll fr = edges[j].fr,to = edges[j].to,w = edges[j].w;
        //     if(last[fr] + w < dis[to])
        //     {
        //         // 有负权回路
        //     }
        // }
}
```

2.3 Dijkstra

```
void dij(int s)
{
    for (int i = 1;i <= n;i++)
        dis[i] = INF;
    dis[s] = 0;
    priority_queue<PII,vector<PII>,greater<PII>>q;
    q.push({0,s});
    while(!q.empty())
    {
        int x = q.top().second;
        q.pop();
        if(vis[x]) continue;
        vis[x] = 1;
        for (auto it : g[x])
        {
            int to = it.to;
            int w = it.w;
```

```
            if(dis[to] > dis[x] + w)
            {
               dis[to] = dis[x] + w;
               q.push({dis[to],to});
            }
        }
    }
}
```

2.4 Floyd

```
void floyd()
{
    for (int k = 1;k <= n;k++)
    {
        for (int i = 1;i <= n;i++)
        {
            for (int j = 1;j <= n;j++)
                dis[i][j] = min(dis[i][k]+dis[k][j],dis[i][j])
        }
    }
}
```

2.5 Floyd 找最小环

```
void floyd()
{
    for (int k = 1;k <= n;k++)
    {
        for (int i = 1;i < k;i++)
        {
            for (int j = 1;j < k;j++)
            {
                if(i!=j)
                    ans = min(ans,dis[i][j] + mp[i][k] + mp[k][j]);
            }
        }
        for (int i = 1;i <= n;i++)
        {
            for(int j = 1;j <= n;j++)
                dis[i][j] = min(dis[i][k]+dis[k][j],dis[i][j]);
        }
    }
}
```

2.6 Kruskal

```
struct edge
{
    int fr,to,w;
```

```
}edges[n];
int find(int x)
{
    return x == f[x] ? x : f[x] = find(f[x]);
}
bool cmp(edge a,edge b)
{
    return a.w < b.w;
}
void kruskal()
{
    for (int i = 1;i <= n;i++)
        f[i] = i;
    sort(edges+1,edges+1+cnt,cmp);
    for (int i = 1;i <= cnt;i++)
    {
        int a = find(edges[i].fr);
        int b = find(edges[i].to);
        if(a != b)
        {
            ans += edges[i].w;
            f[a] = b;
            tmp++;
            if(tmp == n-1)
                return;
        }
    }
    return << orz //不连通
}
```

2.7 Prim

```
void prim()
{
    for(int i = 1;i <= n;i++)
        dis[i] = INF;
    dis[1] = 0;
    int cnt = 0;
    priority_queue<PII,vector<PII>,greater<PII>> q;
    q.push({0,1});
    while(!q.empty())
    {
        int x = q.top().se;
        ll d = q.top().fi;
        q.pop();
        if(vis[x]) continue;
        vis[x] = 1;
        ans += d;
        cnt++;
        for(auto it : g[x])
        {
            int to = it.fi;
            if(dis[to] > it.se)
            {
```

```
                dis[to] = it.se;
                q.push({dis[to],to});
            }
        }
    }
    if(cnt == n)
        cout << ans;
    else
        cout << "orz";
}
```

2.8 SPFA

```
void spfa(int s)
{
    for (int i = 1;i <= n;i++)
        dis[i] = INF;
    dis[s] = 0;
    queue <int> q;
    q.push(s);
    vis[s] = 1;
    while(!q.empty())
    {
        int tmp = q.front();
        q.pop();
        vis[tmp] = 0;
        for (int i = head[tmp];i;i = edges[i].next)
        {
            int to = edges[i].to;
            if(dis[to] > dis[tmp] + edges[i].w)
            {
                dis[to] = dis[tmp] + edges[i].w;
                if(!vis[to])
                {
                    q.push(to);
                    // num[to]++;
                    // if(num[to] >= n)
                    //     return false //有负权回路
                    vis[to] = 1;
                }
            }
        }
    }
}
```

2.9 dfs 判二分图

```
bool dfs(int x,int c)
{
    color[x] = c;
    for (int i = head[x];i;i = next[i])
    {
```

```
        int to = edges[i].to;
        if(color[to])
        {
            if(color[to] == c)
                return false;
        }
        else if(!dfs(to,3 - c))
            return false;
    }
    return true;
}
bool f = true;
for (int i = 1;i <= n;i++)
{
    if(!color[i])
    {
        if(!dfs(i,1))
        {
            f = false;
            break;
        }
    }
}
if(f)
    cout << "YES";
else
    cout << "NO";
```

2.10 匈牙利算法

```
// 一个二分图中的最大匹配数等于这个图中的最小点覆盖数。
int M , N; // M,N分别表示左右两侧集合的元素数量
int Map[n][n]; // 链接矩阵存图
int p[n]; // 记录当前右侧元素所对应的左侧元素
bool vis[n] // 记录右侧元素是否已被访问过

bool match(int i)
{
    for (int j = 1;j <= N;j++)
    {
        if(Map[i][j] && !vis[j])
        {
            vis[j] = true;
            if(p[j] == 0 || match(p[j]))
            {
                p[j] = i;
                return true;
            }
        }
    }
    return false;
}
int Hungarian()
{
```

```
    int cnt = 0;
    for (int i = 1;i <= M;i++)
    {
        memset(vis,0,sizeof(vis));
        if(match(i))
            cnt++;
    }
    return cnt;
}
```

2.11 同余最短路

```
for(int i = 1;i <= n;i++)
    cin >> a[i];
sort(a+1,a+1+n);
for(int i = 0;i < a[1];i++)
{
    for(int j = 2;j <= n;j++)
        g[i].pb({(i + a[j]) % a[1],a[j]});
}
dij();
ll ans = 0;
for(int i = 0;i < a[1];i++)
{
    // 统计ans
}
cout << ans;
```

2.12 哈密顿路径

```
//A Hamiltonian path is a path which visits every vertex exactly once.
//旅行商问题/哈密顿回路/哈密顿路径
// 从0号点出发
memset(dp,0x3f,sizeof(dp));
dp[1][0] = 0;
for(int i = 0;i < (1 << n);i++)
{
    for(int j = 0;j < n;j++)
    {
        if(i >> j & 1)
        {
            for(int k = 0;k < n;k++)
            {
                if(k != j && (i >> k & 1))
                    dp[i][j] = min(dp[i][j],dp[i ^ (1ll << j)][k] + dis[k][j]);
            }
        }
    }
}

int mi = inf;
for(int i = 1;i < n;i++)
```

```
    mi = min(mi,dp[(1ll << n) - 1][i])  // 哈密顿路径;

mi = inf;
for(int i = 1;i < n;i++)
    mi = min(mi,dp[(1ll << n) - 1][i] + dis[i][0]) // 哈密顿回路
```

2.13 并查集判断二分图

```
for (int i = 1;i <= 2 * n;i++)
    fa[i] = i;
int f = 0;
for (int i = 1;i <= n;i++)
{
    int a,b;
    cin >> a >> b;
    merge(a,b+n);
    merge(b,a+n);
}
for (int i = 1;i <= n;i++)
{
    if(find(i) == find(i + n))
        return false;
}
```

2.14 强连通分量

```
stack<int> s;
// ins : 是否在栈中 scc: 每个点所属的强连通分量编号 sum: 强连通分量的数量
int dfn[n],low[n],ins[n],scc[n],cnt,sum;
void tarjan(int x)
{
    low[x] = dfn[x] = ++cnt;
    ins[x] = 1;
    s.push(x);
    for(auto to : g[x])
    {
        if(!dfn[to])
        {
            tarjan(to);
            low[x] = min(low[x],low[to]);
        }
        else if(ins[to])
            low[x] = min(low[x],dfn[to]);
    }
    if(low[x] == dfn[x])
    {
        ++sum;
        while(!s.empty())
        {
            int tmp = s.top();
            s.pop();
            ins[tmp] = 0;
```

```
            scc[tmp] = sum;
            if(tmp == x)
                break;
        }
    }
}

for (int i = 1;i <= n;i++)
{
    if(!dfn[i])
        tarjan(i);
}
```

2.15 找割点

```
void tarjan(int x,int father)
{
    low[x] = dfn[x] = ++cnt;
    int child = 0;
    for(auto to : g[x])
    {
        if(!dfn[to])
        {
            tarjan(to,x);
            low[x] = min(low[x],low[to]);
            if(low[to] >= dfn[x])
                child++;
        }
        else if(to != father)
            low[x] = min(low[x],dfn[to]);
    }

    if((x == father && child >= 2)||(x != father && child >= 1))
    {
        sum++;
        ans[x] = 1;
    }
}
for (int i = 1;i <= n;i++)
{
    if(!dfn[i])
        tarjan(i,i);
}
```

2.16 拓扑排序

```
int in[n],arr[n] // in:入度 arr是排序后的数组
bool topu(int n)
{
    int cnt = 0;
    queue<int> q;
    for (int i = 1;i <= n;i++)
```

```
    {
        if(in[i] == 0)
            q.push(i);
    }
    while(!q.empty())
    {
        int tmp = q.front();
        q.pop();
        arr[++cnt] = tmp;
        for(int i = head[tmp];i;i = edges[i].next)
        {
            int to = edges[i].to;
            in[to]--;
            if(!in[to])
                q.push(to);
        }
    }
    return cnt == n;
}
```

2.17 最长路

```
void spfa(int s)
{
    for (int i = 1;i <= MAXN;i++)
        dis[i] = -INF;
    dis[s] = 0;
    queue <int> q;
    q.push(s);
    vis[s] = 1;
    while(!q.empty())
    {
        int tmp = q.front();
        q.pop();
        vis[tmp] = 0;
        for (int i = head[tmp];i;i = edges[i].next)
        {
            int to = edges[i].to;
            if(dis[to] < dis[tmp] + edges[i].w)
            {
                dis[to] = dis[tmp] + edges[i].w;
                if(!vis[to])
                {
                    q.push(to);
                    vis[to] = 1;
                }
            }
        }
    }
}
```

2.18 点双联通分量

```cpp
void tarjan(int x,int father)
{
    low[x] = dfn[x] = ++cnt;
    s.push(x);
    for(auto to : g[x])
    {
        if(!dfn[to])
        {
            tarjan(to,x);
            low[x] = min(low[x],low[to]);
        }
        else if(to != father)
            low[x] = min(low[x],dfn[to]);
    }
    if(low[x] == dfn[x])
    {
        sum++;
        while(!s.empty())
        {
            int tmp = s.top();
            s.pop();
            scc[tmp] = sum;
            css[sum] = tmp;
            if(tmp == x)
                break;
        }
    }
}
```

2.19 缩点

```cpp
for (int i = 1;i <= n;i++)
{
    for (auto to : g[i])
    {
        if(scc[i] != scc[to])
            add(scc[i],scc[to]);
    }
}
```

2.20 网络流

```cpp
// 网络流
// 最大流
// 最后的残留网络,两点间的反向路径的值就是两点间的实际流量
// EK 算法 O(nm^2) 算法时间复杂度高,只适用于小图
ll flow[n],mp[n][n],pre[n];
ll bfs(int s,int t)
{
    for(int i = 1;i <= n;i++) pre[i] = -1;
    flow[s] = INF;
    pre[s] = 0;
```

```
    queue<int> q;
    q.push(s);
    while(!q.empty())
    {
        int x = q.front();
        q.pop();
        if(x == t) break;
        for(int i = 1;i <= n;i++)
        {
            if(i != s && mp[x][i] > 0 && pre[i] == -1)
            {
                pre[i] = x;
                q.push(i);
                flow[i] = min(flow[x],mp[x][i]);
            }
        }
    }
    if(pre[t] == -1) return -1;
    return flow[t];
}
ll maxflow(int s,int t)
{
    ll ans = 0;
    while(1)
    {
        ll now = bfs(s,t);
        if(now == -1) break;
        int cur = t;
        while(cur != s)
        {
            int fa = pre[cur];
            mp[fa][cur] -= now;
            mp[cur][fa] += now;
            cur = fa;
        }
        ans += now;
    }
    return ans;
}
void solve()
{
    for(int i = 1;i <= m;i++)
    {
        ll a,b,c;
        cin >> a >> b >> c;
        mp[a][b] += c;
    }
    cout << maxflow(s,t);
}

//Dinic O(n*n*m)
struct edge
{
    int to,w,next;
}edges[m << 1];
int n,m,s,t;
```

```
int head[n],cnt = 1;
int now[n],dep[n];
void add(int from,int to,int w)
{
    edges[++cnt].w = w;
    edges[cnt].to = to;
    edges[cnt].next = head[from];
    head[from] = cnt;
}
ll bfs()
{
    for(int i = 1;i <= n;i++) dep[i] = inf;
    dep[s] = 0;
    now[s] = head[s];
    queue<int> q;
    q.push(s);
    while(!q.empty())
    {
        int x = q.front();
        q.pop();
        for(int i = head[x];i;i = edges[i].next)
        {
            int to = edges[i].to;
            if(edges[i].w > 0 && dep[to] == inf)
            {
                q.push(to);
                now[to] = head[to];
                dep[to] = dep[x] + 1;
                if(to == t)
                    return 1;
            }
        }
    }
    return 0;
}
ll dfs(int u,ll sum)
{
    if(u == t) return sum;
    ll now = 0;
    for(int i = now[u];i&&sum;i = edges[i].next)
    {
        now[u] = i;
        int to = edges[i].to;
        if(edges[i].w > 0 && dep[to] == dep[u] + 1)
        {
            ll k = dfs(to,min(sum,edges[i].w));
            if(!k) dep[to] = 0;
            edges[i].w -= k;
            edges[i^1].w += k;
            now += k;
            sum -= k;
        }
    }
    return now;
}
void solve()
```

```
{
    cin >> n >> m >> s >> t;
    for(int i = 1;i <= m;i++)
    {
        int a,b;
        ll c;
        add(a,b,c);
        add(b,a,0);
    }
    ll ans = 0;
    while(bfs())
        ans += dfs(s,inf);
    cout << ans;
}

//ISAP O(n*n*m)
struct edge
{
    int to,next,from;
    ll w;
}edges[m << 1];
int n,m,s,t;
int head[n],cnt = 1;
int now[n],dep[n],pre[n],gap[n];
void add(int from,int to,int w)
{
    edges[++cnt].w = w;
    edges[cnt].from = from;
    edges[cnt].to = to;
    edges[cnt].next = head[from];
    head[from] = cnt;
}
void bfs()
{
    for(int i = 0;i <= n;i++)
    {
        dep[i] = 0;
        gap[i] = 0;
    }
    dep[t] = 1;
    queue<int> q;
    q.push(t);
    while(!q.empty())
    {
        int x = q.front();
        q.pop();
        gap[dep[x]]++;
        for(int i = head[x];i;i = edges[i].next)
        {
            int to = edges[i].to;
            if(edges[i^1].w && !dep[to])
            {
                dep[to] = dep[x] + 1;
                q.push(to);
            }
        }
```

```
    }
}
ll augment()
{
    ll v = t,flow = INF;
    while(v != s)
    {
        int u = pre[v];
        flow = min(flow,edges[u].w);
        v = edges[u].from;
    }
    v = t;
    while(v != s)
    {
        int u = pre[v];
        edges[u].w -= flow;
        edges[u^1].w += flow;
        v = edges[u].from;
    }
    return flow;
}

void ISAP()
{
    bfs();
    ll flow = 0;
    int u = s;
    for(int i = 1;i <= n;i++) now[i] = head[i];
    while(dep[s] <= n)
    {
        if(u == t)
        {
            flow += augment();
            u = s;
        }
        bool ok = 0;
        for(int i = now[u];i;i = edges[i].next)
        {
            int v = edges[i].to;
            if(edges[i].w && dep[v] + 1 == dep[u])
            {
                ok = 1;
                pre[v] = i;
                now[u] = i;
                u = v;
                break;
            }
        }
        if(!ok)
        {
            gap[dep[u]]--;
            if(!gap[dep[u]]) break;
            int mi = n + 10;
            for(int i = head[u];i;i = edges[i].next)
            {
                int v = edges[i].to;
```

```
                if(dep[v] < mi && edges[i].w)
                    mi = dep[v];
            }
            dep[u] = mi + 1;
            gap[dep[u]]++;
            now[u] = head[u];
            if(u != s)
                u = edges[pre[u]].from;
        }
    }
    cout << flow << "\n";
}
void solve()
{
    cin >> n >> m >> s >> t;
    for(int i = 1;i <= m;i++)
    {
        int a,b;
        ll c;
        cin >> a >> b >> c;
        add(a,b,c);
        add(b,a,0);
    }
    ISAP();
}

// 最大闭合子图的权值等于所有正权点之和减最小割
```

2.21 边双联通分量

```
// tarjan求边双
ll n, m, u, v, cnt;
int dfn[100005], low[100005];
vector<PII> g[100005];
stack<int> st;
int mm;
int scc[100005];
int sz[100005];
void tarjan(int x, int las){
    low[x] = dfn[x] = ++cnt;
    st.push(x);
    for (auto it: g[x]){
        if (it.se == (las ^ 1)) continue;
        if (!dfn[it.fi]){
            tarjan(it.fi, it.se);
            low[x] = min(low[x], low[it.fi]);
        }else low[x] = min(low[x], dfn[it.fi]);
    }
    if (dfn[x] == low[x]){
        vector<int> vec;
        scc[x] = ++mm;
        sz[mm]++;
        while (st.top() != x){
            scc[st.top()] = mm;
```

```
            sz[mm]++;
            st.pop();
        }
        st.pop();
    }
}
for(int i = 1;i <= m;i++)
{
    int a,b;
    cin >> a >> b;
    g[a].pb({b,i << 1});
    g[b].pb({a,i << 1|1});
}
for(int i = 1;i <= n;i++)
{
    if(!dfn[i])
        tarjan(i,0);
}
```

2.22 链式前向星

```
struct edge
{
    int to,w,next;
}edges[m];
int head[n],cnt;
void add(int from,int to,int w)
{
    edges[++cnt].w = w;
    edges[cnt].to = to;
    edges[cnt].next = head[from];
    head[from] = cnt;
}
```

3 字符串

3.1 AC 自动机

```
//AC自动机
// 求有多少个不同的模式串在文本串中出现过，两个模式串不同当且仅当题面编号不同O(文本串长度 + 模式串总长度)
int nex[1000005][26],cnt;
int exist[1000005];
int fail[1000005];
int add()
{
    ++cnt;
    memset(nex[cnt],0,sizeof(nex[cnt]));
    exist[cnt] = 0;
    fail[cnt] = 0;
    return cnt;
```

```
}
void init()
{
    cnt = -1;
    add();
}
void insert(string s)
{
    int cur = 0;
    for (auto it : s)
    {
        if(!nex[cur][it - 'a'])
            nex[cur][it - 'a'] = add();
        cur = nex[cur][it - 'a'];
    }
    exist[cur]++;
}
void getfail()
{
    queue<int> q;
    for(int i = 0;i < 26;i++)
    {
        if(nex[0][i])
        {
            fail[nex[0][i]] = 0;
            q.push(nex[0][i]);
        }
    }
    while(!q.empty())
    {
        int now = q.front();
        q.pop();
        for(int i = 0;i < 26;i++)
        {
            if(nex[now][i])
            {
                fail[nex[now][i]] = nex[fail[now]][i];
                q.push(nex[now][i]);
            }
            else
                nex[now][i] = nex[fail[now]][i];
        }
    }
}
int query(string s)
{
    int now = 0;
    int ans = 0;
    for(int i = 0;i < s.size();i++)
    {
        now = nex[now][s[i] - 'a'];
        for(int j = now;j && exist[j] != -1;j = fail[j]) // exist打标记，trie上走过的点
            不再走
        {
            ans += exist[j];
            exist[j] = -1;
```

```
        }
    }
    return ans;
}
void solve()
{
    int n;
    cin >> n;
    for(int i = 1;i <= n;i++)
    {
        string s;
        cin >> s;
        insert(s);
    }
    getfail();
    string s;
    cin >> s;
    cout << query(s);

}



// AC自动机求每个模式串在文本串中出现次数 O(文本串长度 + 模式串总长度)
int nex[200005][26],cnt;
int exist[200005];
int fail[200005];
int in[200005];
int dp[200005];
int ans[200005];
int add()
{
    ++cnt;
    memset(nex[cnt],0,sizeof(nex[cnt]));
    exist[cnt] = 0;
    fail[cnt] = 0;
    return cnt;
}
void init()
{
    cnt = -1;
    add();
}
void insert(string s,int i)
{
    int cur = 0;
    for (auto it : s)
    {
        if(!nex[cur][it - 'a'])
            nex[cur][it - 'a'] = add();
        cur = nex[cur][it - 'a'];
    }
    exist[cur] = i;
}
void getfail()
{
```

```
    queue<int> q;
    for(int i = 0;i < 26;i++)
    {
        if(nex[0][i])
        {
            fail[nex[0][i]] = 0;
            in[fail[nex[0][i]]]++;
            q.push(nex[0][i]);
        }
    }
    while(!q.empty())
    {
        int now = q.front();
        q.pop();
        for(int i = 0;i < 26;i++)
        {
            if(nex[now][i])
            {
                fail[nex[now][i]] = nex[fail[now]][i];
                in[fail[nex[now][i]]]++;
                q.push(nex[now][i]);
            }
            else
                nex[now][i] = nex[fail[now]][i];
        }
    }
}

void query(string s)
{
    int now = 0;
    for(int i = 0;i < s.size();i++)
    {
        now = nex[now][s[i] - 'a'];
        dp[now]++;
    }
}
map<string,int> mp;
string s[200005];
int pos = 0;
void topu()
{
    queue<int> q;
    for(int i = 1;i <= cnt;i++)
    {
        if(!in[i])
            q.push(i);
    }
    while(!q.empty())
    {
        int x = q.front();
        q.pop();
        ans[exist[x]] = dp[x];
        int to = fail[x];
        in[to]--;
        dp[to] += dp[x];
```

```
        if(!in[to])
            q.push(to);
    }
}
void solve()
{
    init();
    int n;
    cin >> n;
    for(int i = 1;i <= n;i++)
    {
        cin >> s[i];
        if(!mp.count(s[i]))
            mp[s[i]] = ++pos;
        insert(s[i],mp[s[i]]);
    }
    getfail();
    string s1;
    cin >> s1;
    query(s1);
    topu();
    for(int i = 1;i <= n;i++)
        cout << ans[mp[s[i]]] << "\n";
}
```

3.2 hash

```
ll get(int l,int r)
{
    return h[r] - h[l-1] * p[r-l+1];
}
p[0] = 1;
for(int i = 1;i <= n;i++)
{
    h[i] = h[i-1]*base+s[i];
    p[i] = p[i-1]*base;
}
```

3.3 kmp

```
// kmp 在s中找p O(n+m)
void getnext() // next[i] : p[0] 到 p[i-1] 中 最长前后缀交集(前后缀不包括整个串)
{
    next[0] = 0,next[1] = 0;
    for(int i = 1;i < p.size();i++)
    {
        int j = next[i];
        while(j && p[i] != p[j])
            j = next[j];
        if(p[i] == p[j]) next[i+1] = j+1;
        else next[i+1] = 0;
    }
```

```
}
void kmp()
{
    int j = 0;
    for(int i = 0;i < s.size();i++)
    {
        while(j && s[i] != p[j])
            j = next[j];
        if(s[i] == p[j])
            j++;
        if(j == p.size())
        {
            //匹配成功
        }
    }
}
```

3.4 字典树

```
//(空间开: 总字符个数 * 字符集大小)
int nex[MAXN][26],cnt;
int exist[MAXN];
void init()
{
    cnt = -1;
    add();
}
int add()
{
    ++cnt;
    memset(nex[cnt],0,sizeof(nex[cnt]));
    exist[cnt] = 0;
    return cnt;
}
void insert(string s)
{
    int cur = 0;
    for (auto it : s)
    {
        if(!nex[cur][it - 'a'])
            nex[cur][it - 'a'] = add();
        cur = nex[cur][it - 'a'];
    }
    // exist[cur] = 1;
}
ll find_pre(string s)
{
    int cur = 0;
    for (auto it : s)
    {
        if(!nex[cur][it - 'a'])
            return false;
        cur = nex[cur][it - 'a'];
    }
```

```
    return exist[cur];
}
```

3.5 马拉车

```
// manacher
// P[i] 是以字符s[i]为中心字符的最长回文串的半径,最大的p[i]-1就是最长回文串长度,这个最长回文串在原字符串的开头位置是(i-p[i])/2;
int p[n << 1];
string s;
string s1 = "$#";
void manacher()
{
    int r = 0,c;
    for(int i = 0;i < s1.size();i++)
    {
        if(i < r) p[i] = min(p[(c << 1) - i],p[c] + c - i);
        else p[i] = 1;
        while(s1[i + p[i]] == s1[i - p[i]]) p[i]++;
        if(p[i] + i > r)
        {
            r = p[i] + i;
            c = i;
        }
    }
}

void solve()
{
    cin >> s;
    for(auto it : s)
    {
        s1 += it;
        s1 += '#';
    }
    s1 += '&';
    manacher();
    int ans = 0;
    for(int i = 0;i < s1.size();i++)
        ans = max(ans,p[i]-1);
    cout << ans;

}
```

4 数学

4.1 MillerRabin

```
//MillerRabin O(logn) 判断一个数是不是素数
srand((unsigned)time(NULL));
ll ksm(ll a,ll b,ll m)
```

```
{
    a %= m;
    ll ans = 1;
    while(b)
    {
        if(b&1) ans = (LL)ans * a % m;
        a = (LL)a * a % m;
        b = b >> 1;
    }
    return ans;
}
bool Miller_Rabin(ll p) {  // 判断素数
  if (p < 2) return 0;
  if (p == 2) return 1;
  if (p == 3) return 1;
  ll d = p - 1, r = 0;
  while (!(d & 1)) ++r, d >>= 1;  // 将d处理为奇数
  for (ll k = 0; k < 10; ++k) {
    ll a = rand() % (p - 2) + 2;
    ll x = ksm(a, d, p);
    if (x == 1 || x == p - 1) continue;
    for (int i = 0; i < r - 1; ++i) {
      x = (LL)x * x % p;
      if (x == p - 1) break;
    }
    if (x != p - 1) return 0;
  }
  return 1;
}
```

4.2 Miller_Rabin + Pollard_Rho 分解质因子

```
// Miller_Rabin + Pollard_Rho 分解质因子
void fac(ll x) {
    if(x < 2)
        return;
    if(Miller_Rabin(x))
    {
        mp[x]++;
        return;
    }
    ll p = Pollard_Rho(x);
    fac(p),fac(x/p);
}
```

4.3 Pollard_Rho

```
//Pollard_Rho 快速找到x的一个因子(非1非自身) O(n^(1/4))
srand((unsigned)time(NULL));
ll Pollard_Rho(ll x) {
  ll s = 0, t = 0;
  ll c = (ll)rand() % (x - 1) + 1;
```

```
  ll step = 0, goal = 1;
  ll val = 1;
  for (goal = 1;; goal <<= 1, s = t, val = 1) {  // 倍增优化
    for (step = 1; step <= goal; ++step) {
      t = ((LL)t * t + c) % x;
      val = (LL)val * abs(t - s) % x;
      if ((step % 127) == 0) {
        ll d = gcd(val, x);
        if (d > 1) return d;
      }
    }
    ll d = gcd(val, x);
    if (d > 1) return d;
  }
}
```

4.4 中国剩余定理

```
// 保证模数两两互质，这个函数返回的是符合方程组的最小正数解，一般要求的正是这个。在 a1 * a2 * a3 * .... * an内存在唯一解
ll CRT(int k) // k 是数组长度
{
    ll n = 1, ans = 0;
    for(int i=0;i<k;i++) n = n * mod[i];
    for(int i=0;i<k;i++)
    {
        ll m = n/mod[i];
        ll b,y;
        exgcd(m,mod[i],b,y);
        ans = (ans + r[i] * m  * b % n) % n;
    }
    return (ans % n + n) % n;
}
```

4.5 区间筛

```
//区间筛 筛[a,b)内的质数
int primes0[1000005],primes1[1000005];
ll prime[1000005];
int cnt = 0;
void segement_sieve(ll a,ll b)
{
    for(ll i = 0;i * i < b;i++) primes0[i] = 1;
    for(ll i = 0;i < b-a;i++) primes1[i] = 1;
    for(ll i = 2;i * i < b;i++)
    {
        if(primes0[i])
        {
            for(ll j = 2 * i;j * j < b;j += i) primes0[j] = 0;
            for(ll j = max(2ll,(a + i - 1) / i) * i;j < b;j += i) primes1[j-a] = 0;
        }
    }
```

```
    for(ll i = 0;i < b-a;i++)
    {
        if(primes1[i]) prime[++cnt] = i+a;
    }
}
```

4.6 卢卡斯定理

```
// 需要先预处理出fact[]，即阶乘
ll inv(ll a,ll p)
{
    return qpow(a,p-2);
}
ll C(ll m,ll n,ll p)
{
    return m < n ? 0 : fact[m] * inv(fact[n],p) % p * inv(fact[m - n],p) % p;
}
ll lucas(ll m,ll n,ll p)
{
    return n == 0 ? 1 % p : lucas(m / p,n / p,p) * C(m % p,n % p,p) % p;
}
```

4.7 因子个数 O(n)

```
int p[n], g[n], cnt; // p为质数集，g[i]为i的因子个数
bool inp[n];
void init(int n) {
    g[1] = 1;
    for (int i = 2; i <= n; ++i) {
        if (!inp[i]) {
            p[++cnt] = i;
            g[i] = 2;
        }
        for (int j = 1; j <= cnt && i * p[j] <= n; ++j) {
            inp[i * p[j]] = true;
            if (i % p[j] == 0) {
                int t = i, o = 0;
                while (t % p[j] ==0) {t /= p[j]; ++o;}
                g[i * p[j]] = g[i] / (o + 1) * (o + 2);
                break;
            } else {
                g[i * p[j]] = g[i] * 2;
            }
        }
    }
}
```

4.8 因子个数 O(nlogn)

```
// g[i] 代表i的因子个数
for (int i = 1; i <= n; i++) {
    for (int j = i; j <= n; j += i) {
        g[j]++;
    }
}
```

4.9 因子和 O(n)

```
vector<int> prim;
int vis[n];
int ans[n],query[n],num[n]; // ans[i] 为 i 的因子和
void euler()//筛法求因子和
{
    ans[1] = 1;
    for(int i = 2;i < n;i++)
    {
        if(!vis[i])
        {
            prim.push_back(i);
            num[i] = ans[i] = i + 1;
        }
        for(int j = 0 ; j < prim.size() && prim[j] <= n / i; j++)
        {
            vis[i*prim[j]] = 1;
            if(i%prim[j]==0)
            {
                num[prim[j] * i] = num[i] * prim[j] + 1;
                ans[prim[j] * i] = ans[i]/num[i]*num[prim[j] * i];
                break;
            }
            else
            {
                ans[prim[j]*i] = ans[i]*(1 + prim[j]);
                num[prim[j]*i] = prim[j] + 1;
            }
        }
    }
}
```

4.10 因子和 O(nlogn)

```
// g[i]代表i的因子和
for(int i = 1;i <= n;i++)
{
    for(int j = i;j <= n;j+=i)
        g[j] += i;
}
```

4.11 威尔逊定理

```
// 威尔逊定理
// m 为 4 时 , (m - 1)! = 2 (mod m)
// m 为 质数 时 , (m - 1)! = m-1 (mod m)
// m 不为 质数 时 , (m - 1)! = 0(mod m)
```

4.12 扩展欧几里得

```
// a > b;
// ax0 + by0 = gcd(a,b);
// 如果参数为负数,同余方程是在模意义下的，取参数在模意义下的最小正数即可。
// ax + by = c 的一个特解是 x1 = x0*c/gcd(a,b)  y1 = y0*c/gcd(a,b);
// dx = b/gcd(a,b) dy = a/gcd(a,b);
// 通解是 x = x1 + s*dx  y = y1 - s*dy;
// 考虑x,y均为正数 L = ceil(-x1+1,dx) <= s <= floor(y1-1,dy) = R;
// 如果L > R,则不存在正整数解
// 否则,存在正整数解,解的个数是R - L + 1,s = L 时可以得到xmin,ymax ,s = R时可以得到xmax,ymin;(取模)
// 判断一个数k是否在gcd过程中出现if(a % b == k % b && k <= a)
// x的最小正整数解 x = (x1 % dx + dx)%dx;
// x的最大负整数解 x = 模数 - x的最小正整数解
ll exgcd(ll a,ll b,ll &x0,ll &y0)
{
    if(b == 0)
    {
        x0 = 1;
        y0 = 0;
        return a;
    }
    ll d = exgcd(b,a%b,y0,x0);
    y0 -= (a / b) * x0;
    return d;
}//求出 x0,y0

ll inv(ll a,ll p)
{
    ll x,y;
    if(exgcd(a,p,x,y) != 1)
        return -1;
    return (x % p + p) % p;
}
```

4.13 整除分块

```
// 除数范围1-n  被除数 : k
// O(sqrt(n))
for (int l = 1,r;l <= n;l = r + 1)
{
    r = k / (k / l);
}
```

4.14 欧拉函数

```cpp
// 欧拉函数 (小于等于n的正整数中与n互质的个数)
int phi(int n) // 求单个点的欧拉函数 O(sqrt(n))
{
    int res=n;
    for(int i=2;i*i<=n;i++)
    {
        if(n%i==0)
        {
            res = res * (i - 1) / i;
            while(n%i==0) n/=i;
        }
    }
    if(n>1) res = res * (n - 1) / n;
    return res;
}
void phi() // 求1-n的欧拉函数 O(n)
{
    phi[1] = 1;
    int cnt = 0;
    for(ll i = 2;i <= n;i++)
    {
        if(!vis[i])
        {
            vis[i] = 1;
            prime[cnt++] = i;
            phi[i] = i-1;
        }
        for(int j = 0;j < cnt;j++)
        {
            if(i * prime[j] > n) break;
            vis[i * prime[j]] = 1;
            if(i % prime[j] == 0)
            {
                phi[i * prime[j]] = phi[i] * prime[j];
                break;
            }
            phi[i * prime[j]] = phi[i] * phi[prime[j]];
        }
    }
}
```

4.15 欧拉筛

```cpp
//欧拉筛
// 每个合数只被他的最小质因子筛掉
// (快速找到每个数最小质因子)
bool isnp[n];
vector<ll> primes;
void init(ll n)
{
    isnp[1] = 1;
    for (ll i = 2; i <= n; i++)
```

```
    {
        if (!isnp[i])
            primes.push_back(i);
        for (ll it : primes)
        {
            if (it * i > n)
                break;
            isnp[it * i] = 1;
            if (i % it == 0)
                break;
        }
    }
}
```

4.16 求 x 的所有因子

```
// 求x的所有因子 时间复杂度 max(sqrt(x),x的因子数)
map<int,int> mp;
for(int i = 2;i * i <= x;i++)
{
    if(x % i == 0)
    {
        while(x % i == 0)
        {
            mp[i]++;
            x /= i;
        }
    }
}
if(x > 1)
    mp[x]++;
vector<PII> v;
for(auto it : mp)
    v.pb({it.fi,it.se});
vector<int> ans;
void dfs(int p,int x)
{
    if(p == v.size())
    {
        ans.pb(x);
        return;
    }
    for(int i = 0;i <= v[p].se;i++)
    {
        dfs(p+1,x);
        x *= it.fi;
    }
}
dfs(0,1);
```

4.17 求区间内和 k 互质数的个数

```cpp
//求区间L R 内与k互质的数的个数
vector<int> primes;
int num = 0;
int sum = 0;
for(ll i = 2;i * i <= n;i++)
{
    if(n%i == 0)
    {
        primes.pb(i);
        while(n%i==0)
            n/=i;
    }
}
if(n > 1)
primes.pb(n);

void dfs(int pos,int mul,int len)
{
    if(pos == primes.size())
    {
        if(len)
        {
            if(len&1)
                sum += num/mul;
            else
                sum -= num/mul;
        }
        return;
    }
    dfs(pos+1,mul,len);
    dfs(pos+1,mul*primes[pos],len+1);
}
int ask(int r,int k)
{
    num = r; sum = 0;
    dfs(0,1,0);
    return r - sum;
}
```

4.18 矩阵

```cpp
// 定义矩阵
struct matrix
{
    ll x[105][105];
    matrix(){memset(x,0,sizeof(x));}
}
// 矩阵乘法 ()
matrix multiply(matrix &a,matrix &b)
{
    matrix c;
    for(int i = 1;i <= n;i++)
    {
        for(int j = 1;j <= n;j++)
```

```
        {
            for(int k = 1;k <= n;k++)
                c.x[i][j] += a.x[i][k] * b.x[k][j];
        }
    }
    return c;
}
// 矩阵快速幂
matrix mksm(matrix &a,ll m)
{
    matrix res;
    for(int i = 1;i <= n;i++)
        res.x[i][i] = 1;
    while(m)
    {
        if(m&1) res = multiply(res,a);
        a = multiply(a,a);
        m = m >> 1;
    }
    return ans;
}
```

4.19 等差乘等比求和

```
形如这样： (a*n+b)*ksm(q,n-1); q != 0 && q != 1
s = (An + B)*ksm(q,n) - B;
A = a * inv(q-1);
B = (b - A) * inv(q-1);
```

4.20 等差数列求和

```
s = n*a1 + n*(n-1)d * inv(2);
```

4.21 等比数列求和

```
s = a1 * (ksm(q,n) - 1) * inv(q - 1);
```

4.22 线性基

```
// 线性基
int cnt = 0; //如果线性基大小小于原数组大小，说明能构造出来异或为0的子集
void ins(ll x)  //构造线性基
{
    for(int i = 60;i >= 0;i--)
    {
        if(x >> i & 1)
        {
            if(!p[i])
```

```
            {
                p[i] = x;
                cnt++;
                break;
            }
            else
                x ^= p[i];
        }
    }
}

ll ask(ll x) //查询元素x能否被异或出来
{
    for(int i = 60;i >= 0;i--)
        if(x >> i & 1) x ^= p[i];
    return x == 0;
}
ll askma() // 查询异或最大值
{
    ll ans = 0;
    for(int i = 60;i >= 0;i--)
        if((ans ^ p[i]) > ans) ans ^= p[i];
    return ans;
}
ll askmi() // 查询异或最小值(记得特判0)
{
    if(zero) return 0;
    for(int i = 0;i <= 60;i++)
    {
        if(p[i])
            return p[i];
    }

}
void rebuild() //重新构造，方便查询异或第k小
{
    cnt = 0;
    for(int i = 60;i >= 0;i--)
    {
        for(int j = i-1;j >= 0;j--)
        {
            if(p[i] >> j & 1)
                p[i] ^= p[j];
        }
    }
    for(int i = 0;i <= 60;i++)
    {
        if(p[i])
            d[cnt++] = p[i];
    }
}
ll kth(ll k) //找异或第k小(记得特判0)
{
    if(k >= (1ll << cnt)) return -1; // 找不到
    ll ans = 0;
    for(int i = 60;i >= 0;i--)
```

```
        if(k >> i & 1) ans ^= d[i];
    return ans;
}
```

4.23 线性求逆元

```
inv[1] = 1;
for(int i = 2; i <= n; ++ i)
    inv[i] = (ll)(p - p / i) * inv[p % i] % p;
```

4.24 组合数

```
fac[0] = 1;
for (int i = 1;i <= 1000;i++)
    fac[i] = (fac[i-1] * i) % mod;
ifac[1000] = ksm(fac[1000], mod - 2);
for (int i = 1000; i >= 1; i--)
    ifac[i - 1] = ifac[i] * i % mod;
ll C(int n, int m)
{
    if (n < m || m < 0)return 0;
    return fac[n] * ifac[m] % mod * ifac[n - m] % mod;
}

ll A(int n, int m)
{
    if (n < m || m < 0)return 0;
    return fac[n] * ifac[n - m] % mod;
}
```

4.25 莫比乌斯函数

```
void mobius()  // 求1-n的莫比乌斯函数 O(n)
{
    int cnt = 0;
    vis[1] = 1;
    mob[1] = 1;
    for(ll i = 2;i <= n;i++)
    {
        if(!vis[i]) prime[cnt++] = i,mob[i] = -1;
        for(int j = 0;j < cnt && prime[j] * i <= n;j++)
        {
            vis[prime[j] * i] = 1;
            mob[i * prime[j]] = (i % prime[j] ? -mob[i] : 0);
            if(i % prime[j] == 0) break;
        }
    }
}
```

4.26 费马小定理

```
//费马小定理
//若p为素数,gcd(a,p) == 1,则 a^(p-1) == 1(mod p)
//费马小定理降幂
//a^x % p == a ^ (x % (p-1)) % p
```

4.27 降幂

```
//费马小定理降幂
//a^x % p == a ^ (x % (p-1)) % p
```

4.28 预处理 log2

```
for (int i = 1;i <= n;i++)
{
    lg[i] = lg[i-1] + (1 << (lg[i-1] + 1) == i);
}
```

5 数据结构

5.1 LCA

```
void dfs(int x,int father) // 求深度
{
    fa[x][0] = father;
    dep[x] = dep[father] + 1;
    for (int i = 1;i <= 20;i++)
        fa[x][i] = fa[fa[x][i-1]][i-1];
    for (auto it : g[x])
    {
        int to = it;
        if(to != father)
            dfs(to,x);
    }
}
int lca(int a, int b)
{
    if (dep[a] > dep[b])
        swap(a, b);
    for(int i = 20;i >= 0;i--)
    {
        if(dep[fa[b][i]] >= dep[a])
            b = fa[b][i];
    }
    if (a == b)
        return a;
    for (int k = 20; k >= 0; k--)
```

```
        if (fa[a][k] != fa[b][k])
            a = fa[a][k], b = fa[b][k];
    return fa[a][0];
}
```

5.2 ST 表

```
for (int i = 1;i <= n;i++)
{
    int x;
    cin >> x;
    ma[i][0] = x;
}
for (int j = 1;j <= 20;j++)
{
    for (int i = 1;i + (1 << j) - 1 <= n;i++)
        ma[i][j] = max(ma[i][j-1],ma[i + (1 <<(j - 1))][j-1]);
}
for(i = 0;i < m;i++)
{
    int l,r;
    cin >> l >> r;
    int s = lg[r - l + 1];
    int ma1 = max(ma[l][s],ma[r - (1 << s)+1][s]);
}
```

5.3 cdq 分治求三维偏序

```
struct node
{
    ll x,y,z,w,ans;
}a[MAXN],b[MAXN];
bool cmpx(node a,node b)
{
    if(a.x != b.x)
        return a.x < b.x;
    else
    {
        if(a.y != b.y)
            return a.y < b.y;
        return a.z < b.z;
    }
}
bool cmpy(node a,node b)
{
    if(a.y != b.y)
        return a.y < b.y;
    return a.z < b.z;
}
ll t[MAXN];
int lowbit(int x)
{
```

```
    return x & (-x);
}
void add(int pos,int x)
{
    for(int i = pos;i < MAXN;i += lowbit(i))
        t[i] += x;
}
ll ask(int pos)
{
    ll ans = 0;
    for(int i = pos;i;i -= lowbit(i))
        ans += t[i];
    return ans;
}
void cdq(int l,int r)
{
    if(l == r) return;
    int mid = l + r >> 1;
    cdq(l,mid),cdq(mid + 1,r);
    sort(b+l,b+1+mid,cmpy);
    sort(b+1+mid,b+1+r,cmpy);
    int i = l;
    for(int j = mid + 1;j <= r;j++)
    {
        while(b[i].y <= b[j].y && i <= mid)
            add(b[i].z,b[i].w),i++;
        b[j].ans += ask(b[j].z);
    }
    for(int j = l;j < i;j++)
        add(b[j].z,-b[j].w);
}
ll cnt[MAXN];
void solve()
{
    int n,k;
    cin >> n >> k;
    for(int i = 1;i <= n;i++)
        cin >> a[i].x >> a[i].y >> a[i].z;
    sort(a+1,a+1+n,cmpx);
    int nn = 0,c = 0;
    for(int i = 1;i <= n;i++) // 去重
    {
        c++;
        if(a[i].x != a[i+1].x || a[i].y != a[i+1].y || a[i].z != a[i+1].z)
            b[++nn] = a[i],b[nn].w = c,c = 0;
    }
    cdq(1,nn);
    for(int i = 1;i <= nn;i++)
        cnt[b[i].ans + b[i].w - 1] += b[i].w;
    for(int i = 0;i < n;i++)
        cout << cnt[i] << '\n';

}
```

5.4 dsu on tree

```cpp
//dsu on tree O(nlogn) (cf 600E)
map<int,int> dfs(int x,int fa)
{
    map<int,int> mp;
    mp[c[x]] = 1;
    mx[x] = 1;
    ans[x] = c[x];
    for(auto to : g[x])
    {
        if(to == fa)
            continue;
        map<int,int> mp2 = dfs(to,x);
        if(mp.size() < mp2.size())
        {
            mx[x] = mx[to];
            ans[x] = ans[to];
            swap(mp,mp2);
        }
        for(auto it : mp2)
        {
            mp[it.fi] += it.se;
            if(mp[it.fi] > mx[x])
                ans[x] = it.fi,mx[x] = mp[it.fi];
            else if(mp[it.fi] == mx[x])
                ans[x] += it.fi;
        }
    }
    return mp;
}
```

5.5 三分

```cpp
ll l = 0,r = 1e18;
while (l < r)
{
    ll lmid = l + (r-l)/3,rmid = r-(r-l)/3;
    if(f(rmid) < f(lmid))
        l = lmid + 1;
    else
        r = rmid - 1;
}
printf("%.10lf",f(l));
```

5.6 主席树前 k 大之和

```cpp
int n,m;
ll a[MAXN];
ll b[MAXN];
int root[MAXN];
int cnt;
```

```
int nn;
struct node
{
    ll l,r,num,sum,s,val;
}t[MAXN << 5];
int update(int pre,int x,ll v,int cl = 1,int cr = nn)
{
    int rt = ++cnt;
    t[rt].l = t[pre].l;
    t[rt].r = t[pre].r;
    t[rt].num = t[pre].num + 1;
    t[rt].sum = t[pre].sum + v;
    int mid = cl + cr >> 1;
    if(cl == cr)
    {
        t[rt].val = v;
        return rt;
    }
    if(cl < cr)
    {
        if(x <= mid)
            t[rt].l = update(t[pre].l,x,v,cl,mid);
        else
            t[rt].r = update(t[pre].r,x,v,mid + 1,cr);
    }
    return rt;
}
ll ask(int l,int r,int k,int cl = 1,int cr = nn)
{
    if(cl == cr)
        return t[r].val * k;
    int x = t[t[r].r].num - t[t[l].r].num;
    int mid = cl + cr >> 1;
    if(x >= k)
        return ask(t[l].r,t[r].r,k,mid+1,cr);
    else
        return t[t[r].r].sum - t[t[l].r].sum + ask(t[l].l,t[r].l,k-x,cl,mid);
}
void solve()
{
    cnt = 0;
    cin >> n;
    for(int i = 1;i <= n;i++)
        cin >> a[i],b[i] = a[i];
    sort(b+1,b+1+n);
    nn = unique(b+1,b+1+n) - b - 1;
    for(int i = 1;i <= n;i++)
    {
        int p = lower_bound(b+1,b+1+nn,a[i]) - b;
        root[i] = update(root[i-1],p,a[i]);
    }
    cin >> m;
    for(int i = 1;i <= m;i++)
    {
        int l,r,k;
        cin >> l >> r >> k;
```

```
        ll ans = ask(root[l-1],root[r],k);
        ll tmp = r - l + 1;
        cout << tmp * (tmp + 1) * (2 * tmp  + 1) / 6 + ans << "\n";
        // cout << ans << "\n";
    }
}
```

5.7 主席树区间第 k 小

```
// 可持久化线段树求区间第k小 多测令cnt = 0
int n,m;
ll a[MAXN];
ll b[MAXN];
int root[MAXN];
int cnt;
int nn;
struct node
{
    int l,r,num;
}t[MAXN << 5];
int update(int pre,int x,int cl = 1,int cr = nn)
{
    int rt = ++cnt;
    t[rt].l = t[pre].l;
    t[rt].r = t[pre].r;
    t[rt].num = t[pre].num + 1;
    int mid = cl + cr >> 1;
    if(cl < cr)
    {
        if(x <= mid)
            t[rt].l = update(t[pre].l,x,cl,mid);
        else
            t[rt].r = update(t[pre].r,x,mid + 1,cr);
    }
    return rt;
}
int ask(int l,int r,int k,int cl = 1,int cr = nn)
{
    if(cl == cr)
        return cl;
    int x = t[t[r].l].num - t[t[l].l].num;
    int mid = cl + cr >> 1;
    if(x >= k)
        return ask(t[l].l,t[r].l,k,cl,mid);
    else
        return ask(t[l].r,t[r].r,k-x,mid + 1,cr);
}
void solve()
{
    cin >> n >> m;
    for(int i = 1;i <= n;i++)
        cin >> a[i],b[i] = a[i];
    sort(b+1,b+1+n);
    nn = unique(b+1,b+1+n) - b - 1;
```

```
    for(int i = 1;i <= n;i++)
    {
        int p = lower_bound(b+1,b+1+nn,a[i]) - b;
        root[i] = update(root[i-1],p);
    }
    for(int i = 1;i <= m;i++)
    {
        int l,r,k;
        cin >> l >> r >> k;
        int t = ask(root[l-1],root[r],k);
        cout << b[t] << "\n";
    }
}
```

5.8 二维 ST 表

```
ll dp[505][505][10][10];
ll a[1005][1005];
void RMQ_init2d(int n,int m)
{
    for(int i = 1;i <= n;i++)
    {
        for(int j = 1;j <= m;j++)
        {
            dp[i][j][0][0] = a[i][j];
        }
    }
    int t = log((double)m) / log(2.0);

    for(int i = 0;i <= t;i++)
    {
        for(int j = 0;j <= t;j++)
        {
            if(i==0&&j==0)
                continue;
            for(int row = 1;row + (1<<i) - 1 <= n;row++)
            {
                for(int col = 1;col + (1<<j) - 1 <= m;col++)
                {
                    if(i)
                        dp[row][col][i][j] = max(dp[row][col][i-1][j],dp[row+(1<<(i-1))
                            ][col][i-1][j]);
                    else
                        dp[row][col][i][j] = max(dp[row][col][i][j-1],dp[row][col+(1<<(j
                            -1))][i][j-1]);
                }
            }
        }
    }
}
ll RMQ_2d(int x1,int y1,int x2,int y2)
{
    int k1 = log(double(x2 - x1 + 1)) / log(2.0);
    int k2 = log(double(y2 - y1 + 1)) / log(2.0);
```

```
    ll m1 = dp[x1][y1][k1][k2];
    ll m2 = dp[x2 - (1<<k1) + 1][y1][k1][k2];
    ll m3 = dp[x1][y2 - (1<<k2) + 1][k1][k2];
    ll m4 = dp[x2 - (1<<k1) + 1][y2 - (1<<k2) + 1 ][k1][k2];
    ll _max = max(max(m1,m2),max(m3,m4));
    return _max;
}
```

5.9 区间异或和最值

```
//nlogn 01字典树优化 trie
int arr[MAXN],num[MAXN];
int nex[MAXN][2],cnt,exist[MAXN];
void insert(int x)
{
    int cur = 0;
    for (int i = 30;i >= 0;i--)
    {
        int u = x >> i & 1;
        if(!nex[cur][u])
            nex[cur][u] = ++cnt;
        cur = nex[cur][u];
        exist[cur] = 1;
    }
}
int query(int x)
{
    int ans = 0,cur = 0;
    for (int i = 30;i >= 0;i--)
    {
        int u = x >> i & 1;
        if(exist[nex[cur][!u]])
        {
            cur = nex[cur][!u];
            ans = ans * 2 + 1;
        }
        else
        {
            cur = nex[cur][u];
            ans = ans * 2;
        }
    }
    return ans;
}
void solve()
{
    int n;
    cin >> n;
    for (int i = 0;i <= cnt;i++)
    {
        nex[i][0] = 0;
        nex[i][1] = 0;
        exist[i] = 0;
    }
```

```
    cnt = 0;
    for (int i = 1;i <= n;i++)
    {
        cin >> arr[i];
        num[i] = num[i-1] ^ arr[i];
    }
    insert(0);
    int ans = 0;
    for (int i = 1;i <= n;i++)
    {
        ans = max(ans,query(num[i]));
        insert(num[i]);
    }
    cout << ans << "\n";
}
```

5.10 单调栈

```
stack<int> s;
for(int i = 1;i <= n;i++)
{
    while(!s.empty()&&arr[s.top()] < arr[i])
    {
        r[s.top()] = i;
        s.pop();
    }
    s.push(i);
}
while(!s.empty())
{
    r[s.top()] = n+1;
    s.pop();
}
```

5.11 单调队列找最值

```
//单调队列找最大值  m: 区间长度 q存储编号
deque<int> q;
for (int i = 1;i <= n;i++)
{
    while(!q.empty()&&i - q.front() >= m)
        q.pop_fornt();
    while(!q.empty()&&arr[q.back()] < arr[i]) //找最小值改成>
        q.pop_back();
    q.push_back(i);
    // if(i >= m)
    //     cout << arr[q.front()];
}
```

5.12 对顶堆

```
if(a[i] > q1.top()) q2.push(a[i]);
else q1.push(a[i]);
while((int)q1.size() - (int)q2.size() > 1)
{
    q2.push(q1.top());
    q1.pop();
}
while((int)q2.size() > (int)q1.size())
{
    q1.push(q2.top());
    q2.pop();
}
```

5.13 带权并查集

```
int f[MAXN],dis[MAXN];
for (int i = 1;i <= n;i++)
    f[i] = i;
int find(int x)
{
    if(x == f[x])
        return x;
    int tmp = find(f[x]);
    dis[x] = dis[f[x]] + dis[x];
    return f[x] = tmp;
}
void merge(int a,int b,int w) // w的大小要具体分析
{
    int x = find(a),y = find(b);
    f[x] = y;
    dis[x] = dis[b] + w - dis[a];
}
```

5.14 并查集

```
for (int i = 1;i <= n;i++)
    f[i] = i;
int find(int x)
{
   return x == f[x] ? x : f[x] = find(f[x]);
}
void merge(int a,int b)
{
    int x = find(a),y = find(b);
    if(x != y)
        f[x] = y;
}
```

5.15 归并排序

```cpp
int arr[n];
int brr[n];
void msort (int l,int r)
{
    int mid = (l + r) >> 1;
    if(l == r)
        return;
    else
    {
        msort(l,mid);
        msort(mid+1,r);
    }
    int i = l;
    int j = mid+1;
    int t = l;

    while(i <= mid && j <= r)
    {
        if(arr[i] > arr[j])
        {
            ans += mid - i + 1;
            brr[t++] = arr[j++];
        }
        else
        {
            brr[t++] = arr[i++];
        }
    }

    while(i <= mid)
    {
            brr[t++] = arr[i++];
    }

    while(j <= r)
    {
            brr[t++] = arr[j++];
    }

    for (int i = l;i < t;i++)
        arr[i] = brr[i];
    return;
}
```

5.16 扫描线求面积并

```cpp
struct node
{
    ll lx,rx,y,tp;
}len[MAXN];
ll cnt;
ll xx[MAXN];
ll tag[MAXN],t[MAXN];
ll m;
```

```
bool cmp(node a,node b)
{
    return a.y < b.y;
};
void pushup(int p,int cl,int cr)
{
    if(tag[p]) t[p] = xx[cr] - xx[cl];
    else t[p] = t[p << 1] + t[p << 1 | 1];
}
void update(int l,int r,int tp,int p = 1,int cl = 1,int cr = m)
{
    if(l <= cl && cr <= r)
    {
        tag[p] += tp;
        pushup(p,cl,cr);
        return;
    }
    if(cl + 1 == cr)
        return;
    int mid = cl + cr >> 1;
    if(l <= mid) update(l,r,tp,p << 1,cl,mid);
    if(r > mid) update(l,r,tp,p << 1 | 1,mid,cr);
    pushup(p,cl,cr);
}
void solve()
{
    int n;
    cin >> n;
    for(int i = 1;i <= n;i++)
    {
        db x1,y1,x2,y2;
        cin >> x1 >> y1 >> x2 >> y2;
        len[++cnt] = {x1,x2,y1,1};
        xx[cnt] = x1;
        len[++cnt] = {x1,x2,y2,-1};
        xx[cnt] = x2;
    }
    sort(xx+1,xx+1+cnt);
    sort(len+1,len+1+cnt,cmp);
    m = unique(xx+1,xx+1+cnt) - xx - 1;
    ll ans = 0;
    for(int i = 1;i <= cnt;i++)
    {
        ans += t[1] * (len[i].y - len[i-1].y);
        int L = lower_bound(xx+1,xx+1+m,len[i].lx) - xx;
        int R = lower_bound(xx+1,xx+1+m,len[i].rx) - xx;
        update(L,R,len[i].tp);
    }
    cout << ans;
}
```

5.17 树状数组

```
// 树状数组
```

```
int t[n];
// 单点修改
void update_n(int pos,int x)
{
    for (int i = pos; i < n;i += lowbit(i))
        t[i] += x;
}
// 前n项和
int ask_n(int n)
{
    int ans = 0;
    for (int i = n;i;i -= lowbit(i))
        ans += t[i];
    return ans;
}
```

5.18 树的直径

```
// dfs求法
void dfs(int x,int fa)
{
    for(auto to : g[x])
    {
        if(to == fa) continue;
        dis[to] = dis[x] + 1;
        if(dis[to] > dis[p]) p = to;
        dfs(to,x);
    }
}
dfs(1,0);
dis[p] = 0;
dfs(p,0);
cout << dis[p] // 树的直径长度
// 如果要求出一条直径上所有的节点，则可以在第二次DFS的过程中，记录每个点的前序节点，即可从直径的一段一路向前，遍历直径上的所有节点
// 树形dp求法
int dp[MAXN];
void dfs(int x,int fa)
{
    for(auto to : g[x])
    {
        if(to == fa) continue;
        dfs(to,x);
        ans = max(ans,dp[x] + dp[to] + 1);
        dp[x] = max(dp[x],dp[to] + 1);
    }
}
```

5.19 树的重心

```
// 树的重心 ：如果在树中选择某个节点并删除，这棵树将分为若干棵子树，统计子树节点数并记录最大值。取遍树上所有节点，使此最大值取到最小的节点被称为整个树的重心。
```

```
//sz : 树的大小 mss: 最大子树大小
vector<int> ctr // 重心
void dfs(int x,int father)
{
    sz[x] = 1 , mss[x] = 0;
    for (int i = head[x];i;i = edges[i].next)
    {
        int to = edges[i].to;
        if(to!=father)
        {
            dfs(to,x);
            sz[x] += sz[to];
            mss[x] = max(mss[x],sz[to]);
        }
    }
    mss[x] = max(mss[x],n - sz[x]);
    if(mss[x] <= n/2)
    {
        ctr.push_back(x);
    }
}
```

5.20 线段树

```
ll tree[n << 2],mark[n << 2],n,m,arr[n];
void build(int p = 1,int cl = 1,int cr = n)
{
    if(cl == cr)
    {
        tree[p] = arr[cl];
        return;
    }
    int mid = (cl + cr) >> 1;
    build(p << 1,cl,mid);
    build(p << 1 | 1,mid+1,cr);
    tree[p] = tree[p << 1] + tree[p << 1 | 1];
}
void push_down(int p,int len)
{
    mark[p << 1] += mark[p];
    mark[p << 1 | 1] += mark[p];
    tree[p << 1] += mark[p] * (len - len / 2);
    tree[p << 1 | 1] += mark[p] * (len / 2);
    mark[p] = 0;
}
void update(int l,int r,int d,int p = 1,int cl = 1,int cr = n)
{
    if(cl >= l && cr <= r)
    {
        tree[p] += (cr - cl + 1) * d;
        mark[p] += d;
        return;
    }
    push_down(p,cr-cl+1);
```

```
    int mid = (cl + cr) >> 1;
    if(mid >= l) update(l,r,d,p << 1,cl,mid);
    if(mid < r) update(l,r,d,p << 1 | 1,mid+1,cr);
    tree[p] = tree[p << 1] + tree[p << 1 | 1];
}
ll ask(int l,int r,int p = 1,int cl = 1,int cr = n)
{
    if(l <= cl && cr <= r)
        return tree[p];
    push_down(p,cr - cl + 1);
    ll mid = (cr + cl) >> 1,ans = 0;
    if(mid >= l) ans += ask(l,r,p << 1 ,cl ,mid);
    if(mid < r) ans += ask(l,r,p << 1 | 1,mid+1,cr);
    return ans;
}
```

5.21 莫队

```
//普通莫队 O(n * sqrt(m))
int belong(int x)
{
    return x/b;
}
bool cmp(qry a,qry b)
{
    if(belong(a.l) == belong(b.l))
    {
        if(belong(a.l) & 1)
            return a.r < b.r;
        else
            return a.r > b.r;
    }
    return belong(a.l) < belong(b.l);
}
void erase(int x)
{

}
void insert(int x)
{

}
    cin >> n >> m >> k;
    b = n / (sqrt(m) + 1) + 1;
    for (int i = 1;i <= n;i++)
        cin >> arr[i];
    for (int i = 1;i <= m;i++)
    {
        cin >> q[i].l >> q[i].r;
        q[i].t = i;
    }
    sort(q+1,q+m+1,cmp);
    int l = 1,r = 0;
    for (int i = 1;i <= m;i++)
```

```
    {
        while(l < q[i].l)
            erase(arr[l]),l++;
        while(l > q[i].l)
            l--,insert(arr[l]);
        while(r < q[i].r)
            r++,insert(arr[r]);
        while(r > q[i].r)
            erase(arr[r]),r--;
        l = q[i].l , r = q[i].r;
        Ans[q[i].t] = ans;
    }
    for (int i = 1;i <= m;i++)
        cout << Ans[i] << "\n";
```

5.22 预处理 LCA

```
// 预处理LCA 预处理(nlogn) 查询(o1)
vector<int> g[MAXN];
int dep[MAXN],mi[MAXN][25];
int idx,dfn[MAXN],que[MAXN],lg[MAXN];
for(int i = 1;i <= idx;i++)
        lg[i] = lg[i-1] + (1 << (lg[i-1] + 1) == i);
void dfs(int x,int father)
{
    dfn[x] = ++idx,que[idx] = x;
    dep[x] = dep[father] + 1;
    for(auto it : g[x])
    {
        int to = it;
        if(to != father)
        {
            dfs(to,x);
            que[++idx] = x;
        }
    }
}
void buildst()
{
    for(int i = 1;i <= idx;i++)
        mi[i][0] = que[i];
    for(int j = 1;j <= 20;j++)
    {
        for(int i = 1;i + (1 << j) - 1 <= idx;i++)
        {
            if(dep[mi[i][j-1]] < dep[mi[i + (1 << (j-1))][j-1]])
                mi[i][j] = mi[i][j-1];
            else
                mi[i][j] = mi[i + (1 << (j-1))][j-1];
        }
    }
}
int lca(int l,int r)
{
```

```
    l = dfn[l],r = dfn[r];
    if(l > r)
        swap(l,r);
    int s = lg[r - l + 1];
    if(dep[mi[l][s]] < dep[mi[r - (1 << s) + 1][s]])
        return mi[l][s];
    else
        return mi[r - (1 << s) + 1][s];
}
```

6 杂

6.1 01 异或期望

```
对于n个非0即1的变量，他们异或的期望在n>0时为1/2 ，反之为0
```

6.2 map

```
//用map构造pair会复制构造造成巨大的性能损失,用move函数进行移动构造。
// pair<map<int,int>,int> p;
// map<int,int> mp;
// int a;
// {mp,a} 应该写成 {move(mp),a};
// pair<map<int,int>,int> tmp = pair<map<int,int>,int> dfs(...) 应写成 auto&&[] = dfs
    (...)
```

6.3 oset

```
template <typename T> using o_set = tree<T, null_type, less<T>, rb_tree_tag,
    tree_order_statistics_node_update>;
o_set<int> s;
// order_of_key() 返回 o_set 中小于x的元素个数
// int a = s.order_of_key(x);
```

6.4 sqrt 向上取整

```
ll sqrt(ll n) {
    ll lo = 0, hi = 2 * std::sqrt(n);
    while (lo < hi) {
        ll x = (lo + hi) / 2;
        if (x * x >= n) {
            hi = x;
        } else {
            lo = x + 1;
        }
    }
    return lo;
```

```
}
```

6.5 上取整下取整

```
template <typename T, typename U>
T ceil(T x, U y) {
  return (x > 0 ? (x + y - 1) / y : x / y);
}
template <typename T, typename U>
T floor(T x, U y) {
  return (x > 0 ? x / y : (x - y + 1) / y);
}
```

6.6 二分图

```
//二分图最小点覆盖：给定一张二分图，求出一个最小的点集S，使得图中任意一条边都有至少一个端点属于S。
//Konig定理：二分图最小点覆盖包含的点数等于二分图最大匹配包含的边数。

//二分图最大独立集：选取尽可能多的点使得点集中所有点两两之间无边相连。
//定理：最大独立集 = n - 最大匹配数（n为图的节点个数
```

6.7 全排列

```
//全排列
do
{

}while(next_permutation(v.begin(),v.end()));
```

6.8 判断一个数是不是二的幂次

```
//判断t是不是2的幂次
int t;
if(t & (t-1) == 0)
    cout << "YES";
else
    cout << "NO"
```

6.9 博弈 1

```
// n堆Nim博弈 异或和为0，先手必败
// 阶梯博弈(Nim博弈变种) 奇数节点异或和为0，先手必败
// 树上阶梯博弈和阶梯博弈同理

// SG 定理
```

```
// SG(x) == 0 x为必败点,否则为必胜点
// SG(x) = mex{SG(y) | x can go to y} // y 是小状态 x 是大状态
// 游戏的和的SG值是他们SG值的xor
```

6.10 去重

```
//去重后的范围 1 到 n
// 数组
sort(a+1,a+1+n);
int n = unique(a+1,a+1+n) - a;
n--;
// vector
sort(v.begin(),v.end());
int n = unique(v.begin(),v.end()) - v.begin();
n--;
```

6.11 异或

```
//异或性质
//a - b <= a ^ b <= a + b
```

6.12 期望 1

```
// 如果一个事件的结果A发生的概率是P，则一直做这件事直到第一次发生结果A的期望X是1/p(满足几何分布)
// 呈几何分布的期望是1/p,方差是 (1-p)/(p*p)
```

6.13 矩形相交

```
//判断两个矩形是否相交
// 两个矩形宽高分别为w1,h1,w2,h2;
// 两个矩形中点间的水平距离和竖直距离分别为 w,h
// if(w < (w1 + w2)/2 && h < (h1 + h2) / 2) 则两个矩形相交,否则不相交
```

6.14 规律 1

```
// 1^2 + 2^2 + ... + n^2 = n(n+1)(2n+1)/6
// 1^3 + 2^3 + ... + n^3 = (n(n+1)/2)^2
```

6.15 质数差

```
// 任意一个偶数x,一定存在一对质数差为x;
```

6.16 随机模数

```cpp
typedef long long ll;
using ll = long long;
template<class T>
constexpr T power(T a, ll b) {
    T res {1};
    for (; b; b /= 2, a *= a) {
        if (b % 2) {
            res *= a;
        }
    }
    return res;
}

constexpr ll mul(ll a, ll b, ll p) {
    ll res = a * b - ll(1.L * a * b / p) * p;
    res %= p;
    if (res < 0) {
        res += p;
    }
    return res;
}

template<ll P>
struct MInt {
    ll x;
    constexpr MInt() : x {0} {}
    constexpr MInt(ll x) : x {norm(x % getMod())} {}

    static ll Mod;
    constexpr static ll getMod() {
        if (P > 0) {
            return P;
        } else {
            return Mod;
        }
    }
    constexpr static void setMod(ll Mod_) {
        Mod = Mod_;
    }
    constexpr ll norm(ll x) const {
        if (x < 0) {
            x += getMod();
        }
        if (x >= getMod()) {
            x -= getMod();
        }
        return x;
    }
    constexpr ll val() const {
        return x;
    }
    constexpr MInt operator-() const {
        MInt res;
        res.x = norm(getMod() - x);
```

```
        return res;
    }
    constexpr MInt inv() const {
        return power(*this, getMod() - 2);
    }
    constexpr MInt &operator*=(MInt rhs) & {
        if (getMod() < (1ULL << 31)) {
            x = x * rhs.x % int(getMod());
        } else {
            x = mul(x, rhs.x, getMod());
        }
        return *this;
    }
    constexpr MInt &operator+=(MInt rhs) & {
        x = norm(x + rhs.x);
        return *this;
    }
    constexpr MInt &operator-=(MInt rhs) & {
        x = norm(x - rhs.x);
        return *this;
    }
    constexpr MInt &operator/=(MInt rhs) & {
        return *this *= rhs.inv();
    }
    friend constexpr MInt operator*(MInt lhs, MInt rhs) {
        MInt res = lhs;
        res *= rhs;
        return res;
    }
    friend constexpr MInt operator+(MInt lhs, MInt rhs) {
        MInt res = lhs;
        res += rhs;
        return res;
    }
    friend constexpr MInt operator-(MInt lhs, MInt rhs) {
        MInt res = lhs;
        res -= rhs;
        return res;
    }
    friend constexpr MInt operator/(MInt lhs, MInt rhs) {
        MInt res = lhs;
        res /= rhs;
        return res;
    }
    friend constexpr std::istream &operator>>(std::istream &is, MInt &a) {
        ll v;
        is >> v;
        a = MInt(v);
        return is;
    }
    friend constexpr std::ostream &operator<<(std::ostream &os, const MInt &a) {
        return os << a.val();
    }
    friend constexpr bool operator==(MInt lhs, MInt rhs) {
        return lhs.val() == rhs.val();
    }
```

```
    friend constexpr bool operator!=(MInt lhs, MInt rhs) {
        return lhs.val() != rhs.val();
    }
    friend constexpr bool operator<(MInt lhs, MInt rhs) {
        return lhs.val() < rhs.val();
    }
};

template<>
ll MInt<0>::Mod = 998244353;

constexpr ll P = ll(1E18) + 9;
using Z = MInt<P>;

std::mt19937_64 rng(std::chrono::steady_clock::now().time_since_epoch().count());
const Z B = rng();
```